

Sprint Report — Fine-tune & Host LLM (lần đầu tay)

Người thực hiện: Dương Phương Thảo **Thời gian:** 2 tuần (chốt sprint ngày 06/06/2026) **Dự án:** Fine-tune Qwen2.5-1.5B để viết lại câu lệnh hội thoại tiếng Việt cho trợ lý trong xe

1. Tổng quan

Dạy một mô hình nhỏ (**Qwen2.5-1.5B-Instruct**) đọc đoạn hội thoại nhiều lượt rồi **viết lại thành một câu lệnh duy nhất, ngắn gọn, đủ ý** (ví dụ: "Gọi cho mẹ đi" + "À mà 7 giờ tối nhé" + "Đừng gọi bố" → một câu chuẩn). Đây là **lần đầu mình làm trọn vòng đời fine-tune**: từ chuẩn bị dữ liệu → huấn luyện → host mô hình lên server (vLLM) → đo lường chất lượng → tìm điểm yếu → cải thiện. Phần lớn giá trị nằm ở việc đi hết được vòng lặp này và hiểu *tại sao* từng bước lại cần thiết.

2. Mình đã làm những gì

a) Chuẩn bị dữ liệu

- Sinh tổng cộng **20.000 mẫu** bằng template (ghép các tình huống quen thuộc: bật/tắt điều hòa, phát nhạc, chỉ đường, gọi điện...), chia thành **train 16.001 / valid 2.000 / test 1.999**. Sau khi vá thêm 1.000 mẫu có chủ đích, tập train tăng lên **17.001** (train_v2).
- Dùng các **Frontier Models** (ChatGPT, Gemini, Claude, Grok — chạy trên browser) để tạo riêng một **bộ benchmark khó gồm 1.044 mẫu** với 8 dạng tình huống dễ sai: đại từ mơ hồ, ngữ cảnh nhiễu, slot nằm rải nhiều lượt, sửa lời, trộn Việt-Anh, phủ định... Bộ này dùng để chấm điểm, **không** cho mô hình thấy lúc train.

b) Fine-tune lần đầu

- Dùng kỹ thuật **LoRA (cụ thể là QLoRA, 4-bit)**. Có thể hình dung LoRA như việc *dán thêm một miếng vá nhỏ* lên mô hình gốc thay vì huấn luyện lại toàn bộ — nhờ vậy chỉ cần một GPU T4 16GB miễn phí trên Kaggle là chạy được, và "miếng vá" sinh ra chỉ nặng ~50MB.
- Đây là rào cản đầu tiên mình vượt qua: hiểu được vì sao LoRA giúp người ít tài nguyên vẫn fine-tune được mô hình.

c) Host mô hình và làm demo

- Dựng **vLLM** để phục vụ mô hình qua một API chuẩn (giống OpenAI), rồi dùng **ngrok** để mở một đường truy cập từ xa — nhờ vậy máy local hoặc người khác đều gọi tới được.
- Làm một **giao diện demo bằng Gradio** kiểu "bấm-là-chạy": nạp sẵn nhiều mẫu test thật, click vào là thấy ngay mô hình viết lại thế nào so với đáp án chuẩn.

d) Đo lường chất lượng

- Xây pipeline benchmark dùng một mô hình khác làm **"giám khảo" (LLM-judge)** để chấm 4 tiêu chí: giữ đúng ý định, đủ thông tin (slot), không bịa thêm, và giữ đúng phủ định.

- Một quyết định quan trọng: **tách rời bước chạy mô hình (predict) và bước chấm (judge)** thành hai bước độc lập, và **giữ nguyên một giám khảo cố định** xuyên suốt các phiên bản để so sánh công bằng.

e) Vòng lặp cải thiện

Thay vì train một lần rồi dừng, mình lặp lại chu trình: **đo** → **tìm điểm yếu** → **vá dữ liệu đúng chỗ** → **train lại** → **đo lại**. Trong sprint này mình đi được 2 vòng lớn (v1.0 → v2.0) và vài bước chần đoán ở giữa.

3. Kết quả chính

Bảng 1 — Fine-tune so với mô hình gốc (trên benchmark 1.044 mẫu):

Chỉ số	Base (gốc)	LoRA (fine-tune)	Chênh lệch
Command Accuracy (chính)	52.8%	65.9%	+13.1
Intent Accuracy	81.0%	88.5%	+7.5
Slot Completeness	58.7%	70.9%	+12.2
Multi-turn slot (điểm yếu nhất)	13.5%	52.8%	+39.3
Tỉ lệ bịa thông tin (càng thấp càng tốt)	11.8%	7.9%	-3.9 ✓

→ Fine-tune giúp **+13 điểm** ở chỉ số chính, và cách biệt lớn nhất nằm ở **multi-turn slot** (gộp thông tin rải qua nhiều lượt) — đúng chỗ cần kiến thức riêng của tác vụ.

Bảng 2 — Đã cải thiện được gì sau khi vá dữ liệu (LoRA v1.0 → v2.0, thêm ~1.000 mẫu B2/B3):

Dạng câu khó	Trước vá (v1.0)	Sau vá (v2.0)	Thay đổi
multi_turn_slot (mục tiêu vá)	52.8%	57.1%	+4.3 ✓
implicit_reference (hưởng lợi)	65.9%	69.0%	+3.1 ✓
Command Accuracy tổng	65.9%	65.6%	-0.3 (phẳng)
Phủ định (đã hiệu chỉnh)	81.5%	81.5%	0.0 (chưa giải quyết)

→ Bản vá **đánh trúng đích**: kéo điểm yếu nhất lên **+4.3** mà tổng thể **không bị hỏng** — đúng tinh thần một bản vá "phẫu thuật" thay vì nhồi data tràn lan.

- Một phát hiện đáng nhớ**: mô hình 1.5B có xu hướng **"nén" câu quá tay** — nó hay bỏ rơi mệnh đề phủ định ("đừng gọi...", "không bật...") và làm rớt một số chi tiết. Quan trọng hơn, mình xác nhận được đây là **lỗi thật của mô hình (~-6.7 điểm)**, không phải lỗi đo lường.

4. Mình học được gì

Đây là phần mình thấy giá trị nhất, vì là sprint fine-tune đầu tay.

- **Những kỹ năng mới lần đầu được làm.** Qua sprint này mình học được cách *fine-tune* một mô hình trên Kaggle (tận dụng GPU miễn phí), cách *fine-tune* bằng LoRA với LLaMA Factory, cách *research* để chọn mô hình phù hợp với bài toán (so sánh kích thước, năng lực, chi phí), và cách *research* các *metric* để biết nên đo cái gì cho đúng với tác vụ rewrite.
- **Prompt cho judge và base model phải thật chi tiết — và nên thêm few-shot.** Mình học được rằng chất lượng đầu ra phụ thuộc rất nhiều vào việc viết prompt rõ ràng, đầy đủ ràng buộc, và đặc biệt là *cho thêm vài ví dụ mẫu (few-shot)* ngay trong prompt. Với *judge model* (mô hình chấm điểm), prompt chi tiết giúp chấm nhất quán và đáng tin hơn; với *base model* (mô hình chưa fine-tune), few-shot giúp nó hiểu định dạng mong muốn và làm tốt hơn hẳn. Đây là đòn bẩy lớn mà tốn ít công.
- **Mượn sức của mô hình lớn để giúp mô hình nhỏ làm việc hiệu quả hơn.** Một ý tưởng mình thấy rất hay: dùng *mô hình mạnh, thông minh* (như các frontier model) để *tạo ra few-shot và dữ liệu chất lượng cao*, rồi cho mô hình nhỏ học và làm theo. Thay vì kỳ vọng mô hình 1.5B tự giỏi, mình để mô hình lớn "dạy mẫu" — đây chính là cách bộ benchmark khó và các bản vá dữ liệu được tạo ra, và là cách hiệu quả để nâng chất lượng mô hình nhỏ.
- **Train xong chưa phải là xong — đo lường đúng quan trọng ngang việc train.** Lúc đầu chỉ số phủ định tụt tụt -10.1 điểm, mình suýt kết luận mô hình hỏng nặng. Nhưng khi xem kỹ cách chấm điểm, mình nhận ra một phần là do *cách đo quá khắt khe*. Sau khi sửa lại tiêu chí cho hợp lý, con số thật là -6.7. Bài học: nếu không tin tưởng được thước đo thì mọi kết luận sau đó đều lung lay.
- **Chẩn đoán trước khi vá, đừng đoán mò.** Trước khi đổ lỗi cho dữ liệu, mình bỏ công kiểm tra và xác nhận dữ liệu huấn luyện *sạch*. Nhờ vậy mình biết vấn đề nằm ở *hành vi của mô hình* chứ không phải nhầm sai — và chọn đúng hướng sửa.
- **Vá hẹp, đúng chỗ tốt hơn nhồi thêm dữ liệu bừa.** Chỉ ~1.000 mẫu được thiết kế cẩn thận cho đúng kiểu lỗi đã quan sát được lại hiệu quả hơn nhiều so với việc đổ thêm thật nhiều dữ liệu chung chung. Chất lượng thắng số lượng.
- **Hạ tầng cũng là một phần của công việc.** Có những lúc tắc không phải vì mô hình mà vì kỹ thuật: phải nâng giới hạn độ dài ngữ cảnh (2048 → 8192 token) thì bài test mới chạy được; phải tách bước predict/judge và cho phép *chạy tiếp khi bị timeout* thì việc đo mới ổn định; phải giữ giám khảo cố định thì so sánh giữa các phiên bản mới công bằng.
- **Mô hình nhỏ có "trần" năng lực.** Dù dữ liệu sạch và đã vá, khoảng cách ở phủ định vẫn tồn tại. Điều này cho thấy 1.5B có giới hạn với những câu cần suy luận tinh tế — và đôi khi giải pháp không phải thêm dữ liệu mà là dùng mô hình lớn hơn.
- **Làm việc có phiên bản và ghi report từng bước giúp không bị lạc.** Mỗi cải tiến mình ghi thành một báo cáo riêng (v1.0, v1.1, v1.2...). Nhờ vậy lúc nào cũng biết mình đang ở đâu, đã thử gì, và bước tiếp theo là gì.

5. Bước tiếp theo

- **Patch B4:** thêm một bộ mẫu mới dạy mô hình *giữ lại mệnh đề phủ định* và các chi tiết quan trọng thay vì nén bỏ, rồi train lại và đo lại.

- **Nếu phủ định vẫn kẹt:** cân nhắc nâng lên mô hình lớn hơn (Qwen2.5-3B) để vượt qua giới hạn năng lực của bản 1.5B.
-

Tóm lại, sau 2 tuần mình đã đi trọn được một vòng fine-tune end-to-end lần đầu tiên: từ dữ liệu, huấn luyện, host vLLM, đến đo lường và cải thiện có phương pháp. Quan trọng nhất là mình học được cách làm việc một cách có hệ thống — đo cho đúng, chẩn đoán trước khi sửa, và cải thiện từng chút một thay vì đoán mò.